

1. Roles

- **Host:** Users who rent out their properties on the Airbnb platform.
- **User:** Users who search for accommodations and potentially book them on the Airbnb platform.
- **Administrator:** Users who have access to all data and functions and manage the Airbnb platform.

2. Actions

Hosts:

- **Create a listing for their property:** add new listings to the platform with details such as location, description, price, amenities, and house rules.
- **Update the details of their listing:** modify existing listings, including updating descriptions, prices, amenities, neighborhood, location, house rules and cleaning schedule.
- **Manage their availability calendar:** set and update the availability of their properties, including marking dates as available or unavailable.
- **Receive payments for rentals:** receive and track payments for bookings made by guests.
- **Rate guests based on their personal experiences:** leave reviews, complaints and ratings for guests after a stay.
- **View guest reviews:** entering guest reviews and access booking history.
- **Send messages to guests:** send direct messages to guests to clarify questions related to payments and bookings.

Guests:

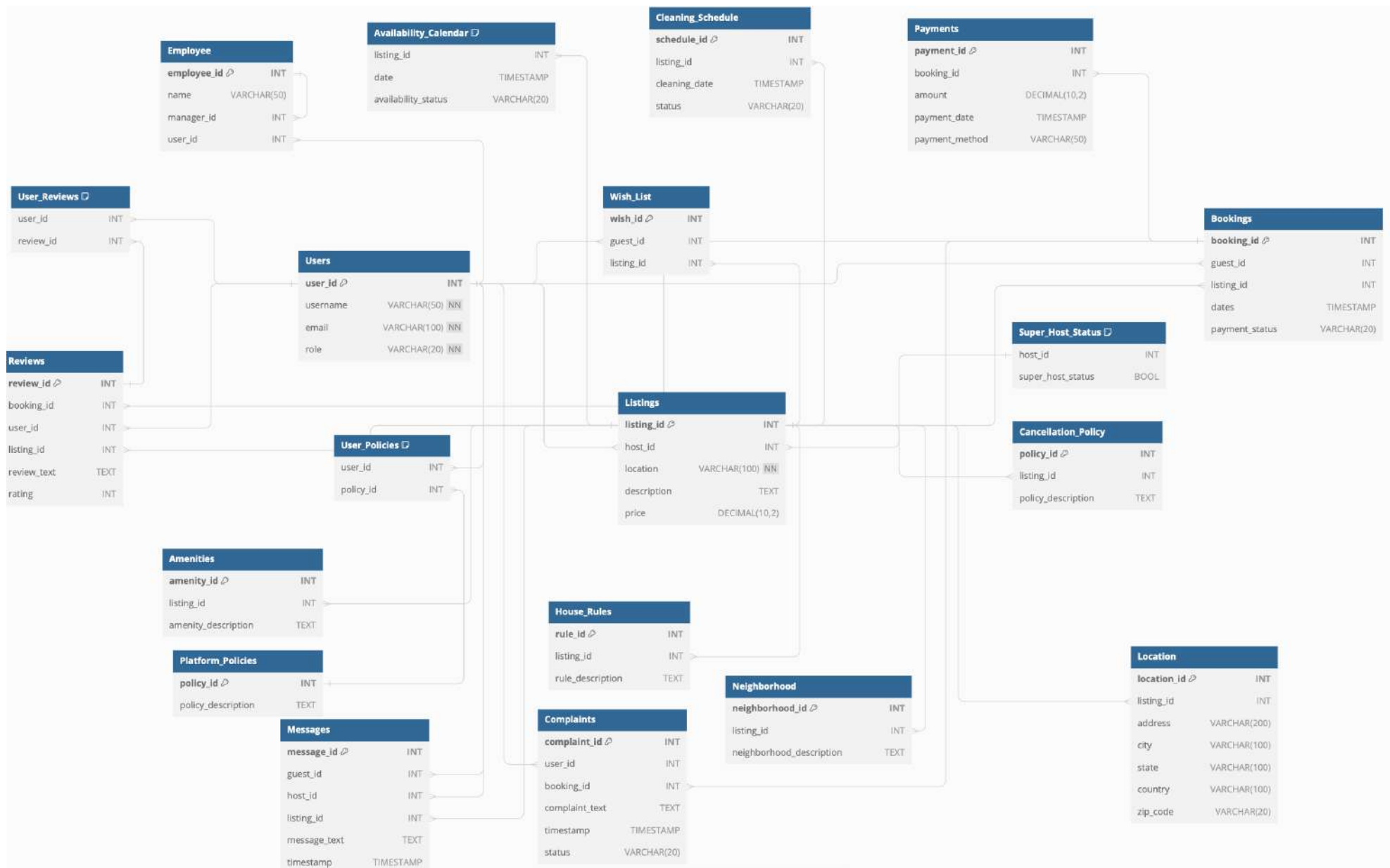
- **Search for accommodations:** search filters to find suitable accommodations based on location, dates, price, and preferences.
- **Create wish lists for accommodations:** saving distinct listings for potential booking.
- **View details of listings:** view information about listing, including photos, amenities, house rules, neighborhood, location, and reviews.
- **Make bookings and payments:** book accommodations and make payments through the platform, including selecting dates, entering payment details, and confirming bookings.
- **Create reviews based on their personal experiences:** leave reviews, complaints and ratings for hosts and properties after a stay.
- **Send messages to hosts:** send direct messages to hosts to clarify questions related to accommodations.

Admins:

- **Manage user accounts:** create, update, and delete user accounts, including managing roles (Host, User, Admin) and user details.
- **Monitor and enforce platform policies:** oversee compliance with platform policies and ensure that users adhere to terms and condition and update cancellation policy.

3. Data and Functions

- **Required Data:** Bookings: booking_id (Primary Key), guest_id, listing_id, dates, payment_status.
- **Functions:**
 - Insert new bookings.
 - Process and record payments.
 - Update booking status and payment details.
- **Required Data:** Reviews: review_id (Primary Key), booking_id, user_id, listing_id, review_text, rating.
- **Functions:**
 - Insert new reviews (guests and hosts).
 - Query and display reviews for listings and users.
- **Required Data:** Availability_calendar: listing_id, date, availability_status.
- **Functions:**
 - Insert and update availability status.
 - Query availability for specific dates and listings.
- **Required Data:** Listings: listing_id (Primary Key), host_id, location, description and price.
- **Functions:**
 - Insert and update listings.
 - Ensure listing compliance through monitoring and reporting.
 - Query availability for details about accommodations and listing via many-to-many relations including Amenities, Neighborhood, House_Rules, Location.
- **Required Data:** Payments: payment_id (Primary Key), booking_id, amount, payment_date, payment_method.
- **Functions:**
 - Insert and update payment status.
 - Ensure payment monitoring for guests, hosts and admins.
- **Required Data:** Employees: employee_id (Primary Key), name, manager_id, user_id.
- **Functions:**
 - Insert and update new employees.
 - Manage employee hierarchy.
- **Required Data:** Users: user_id (Primary Key), username, email, role.
- **Functions:**
 - Insert and update new users.
 - Query availability for details about hosts, guests and admins via many-to-many relations including Reviews, Listings, Bookings, Complaints, Wish_List, and Employee for in-depth queries.



	Column Name	Data Type	Constraints	Description
Table: Users	user_id	INT	PRIMARY KEY	Unique identifier for each user.
	username	VARCHAR(50)	NOT NULL	Username of the user.
	email	VARCHAR(100)	NOT NULL	Email address of the user.
	role	VARCHAR(20)	NOT NULL	Role of the user (e.g., guest, host).
Table: Employee	employee_id	INT	PRIMARY KEY	Unique identifier for each employee.
	name	VARCHAR(50)		Name of the employee.
	manager_id	INT		ID of the employee's manager.
	user_id	INT	FOREIGN KEY	User ID associated with the employee.
Table: Listings	listing_id	INT	PRIMARY KEY	Unique identifier for each listing.
	host_id	INT	FOREIGN KEY	User ID of the host.
	location	VARCHAR(100)	NOT NULL	Location of the listing.
	description	TEXT		Description of the listing.
	price	DECIMAL(10,2)		Price per night for the listing.
Table: Bookings	booking_id	INT	PRIMARY KEY	Unique identifier for each booking.
	guest_id	INT	FOREIGN KEY	User ID of the guest.
	listing_id	INT	FOREIGN KEY	Listing ID for the booking.
	dates	TIMESTAMP		Dates of the booking.
	payment_status	VARCHAR(20)		Status of the payment (e.g., paid, pending).
Table: Reviews	review_id	INT	PRIMARY KEY	Unique identifier for each review.
	booking_id	INT	FOREIGN KEY	Booking ID associated with the review.
	user_id	INT	FOREIGN KEY	User ID of the reviewer.
	listing_id	INT	FOREIGN KEY	Listing ID for the review.
	review_text	TEXT		Text of the review.
Table: Availability_Calendar	rating	INT		Rating given in the review.
	listing_id	INT	FOREIGN KEY	Listing ID for the availability record.
	date	TIMESTAMP		Date of the availability.
	availability_status	VARCHAR(20)		Availability status (e.g., available, booked).
Table: Platform_Policies	policy_id	INT	PRIMARY KEY	Unique identifier for each policy.
	policy_description	TEXT		Description of the policy.
Table: Super_Host_Status	host_id	INT	FOREIGN KEY	User ID of the host.
	super_host_status	BOOL		Status of being a super host.
Table: User_Reviews	user_id	INT	FOREIGN KEY	User ID associated with the review.
	review_id	INT	FOREIGN KEY	Review ID associated with the user.
Table: User_Policies	user_id	INT	FOREIGN KEY	User ID associated with the policy.
	policy_id	INT	FOREIGN KEY	Policy ID associated with the user.
Table: Cancellation_Policy	policy_id	INT	FOREIGN KEY	Unique identifier for each cancellation policy.
	listing_id	INT	FOREIGN KEY	Listing ID associated with the policy.
	policy_description	TEXT		Description of the cancellation policy.
Table: Wish_List	wish_id	INT	PRIMARY KEY	Unique identifier for each wish list entry.
	guest_id	INT	FOREIGN KEY	User ID of the guest.
	listing_id	INT	FOREIGN KEY	Listing ID added to the wish list.
Table: Amenities	amenity_id	INT	PRIMARY KEY	Unique identifier for each amenity.
	listing_id	INT	FOREIGN KEY	Listing ID associated with the amenity.
	amenity_description	TEXT		Description of the amenity.
Table: House_Rules	rule_id	INT	PRIMARY KEY	Unique identifier for each house rule.
	listing_id	INT	FOREIGN KEY	Listing ID associated with the house rule.
	rule_description	TEXT		Description of the house rule.
Table: Neighborhood	neighborhood_id	INT	PRIMARY KEY	Unique identifier for each neighborhood.
	listing_id	INT	FOREIGN KEY	Listing ID associated with the neighborhood.
	neighborhood_description	TEXT		Description of the neighborhood.
Table: Location	location_id	INT	PRIMARY KEY	Unique identifier for each location.
	listing_id	INT	FOREIGN KEY	Listing ID associated with the location.
	address	VARCHAR(200)		Address of the listing.
	city	VARCHAR(100)		City of the listing.
	state	VARCHAR(100)		State of the listing.
	country	VARCHAR(100)		Country of the listing.
Table: Messages	zip_code	VARCHAR(20)		ZIP code of the listing.
	message_id	INT	PRIMARY KEY	Unique identifier for each message.
	guest_id	INT	FOREIGN KEY	User ID of the guest.
	host_id	INT	FOREIGN KEY	User ID of the host.
	listing_id	INT	FOREIGN KEY	Listing ID associated with the message.
	message_text	TEXT		Text of the message.
Table: Payments	timestamp	TIMESTAMP		Timestamp of the message.
	payment_id	INT	PRIMARY KEY	Unique identifier for each payment.
	booking_id	INT	FOREIGN KEY	Booking ID associated with the payment.
	amount	DECIMAL(10,2)		Amount of the payment.
	payment_date	TIMESTAMP		Date of the payment.
Table: Complaints	payment_method	VARCHAR(50)		Method of the payment (e.g., credit card, PayPal).
	complaint_id	INT	PRIMARY KEY	Unique identifier for each complaint.
	user_id	INT	FOREIGN KEY	User ID of the complainant.
	booking_id	INT	FOREIGN KEY	Booking ID associated with the complaint.
	complaint_text	TEXT		Text of the complaint.
	timestamp	TIMESTAMP		Timestamp of the complaint.
Table: Cleaning_Schedule	status	VARCHAR(20)		Status of the complaint (e.g., pending, resolved).
	schedule_id	INT	PRIMARY KEY	Unique identifier for each cleaning schedule.
	listing_id	INT	FOREIGN KEY	Listing ID associated with the cleaning schedule.
	cleaning_date	TIMESTAMP		Date of the cleaning.
	status	VARCHAR(20)		Status of the cleaning (e.g., scheduled, completed).

References (Foreign Keys)			
From Table	From Column	To Table	To Column
Employee	manager_id	Employee	employee_id
Employee	user_id	Users	user_id
Listings	host_id	Users	user_id
Bookings	guest_id	Users	user_id
Bookings	listing_id	Listings	listing_id
Reviews	booking_id	Bookings	booking_id
Reviews	user_id	Users	user_id
Reviews	listing_id	Listings	listing_id
Super_Host_Status	host_id	Listings	host_id
User_Listings	listing_id	Listings	listing_id
User_Reviews	user_id	Users	user_id
User_Reviews	review_id	Reviews	review_id
User_Policies	user_id	Users	user_id
User_Policies	policy_id	Platform_Policies	policy_id
Cancellation_Policy	listing_id	Listings	listing_id
Wish_List	guest_id	Users	user_id
Wish_List	listing_id	Listings	listing_id
Amenities	listing_id	Listings	listing_id
House_Rules	listing_id	Listings	listing_id
Neighborhood	listing_id	Listings	listing_id
Location	listing_id	Listings	listing_id
Messages	guest_id	Users	user_id
Messages	host_id	Users	user_id
Messages	listing_id	Listings	listing_id
Availability_Calendar	listing_id	Listings	listing_id
Payments	booking_id	Bookings	booking_id
Complaints	user_id	Users	user_id
Complaints	booking_id	Bookings	booking_id
Cleaning_Schedule	listing_id	Listings	listing_id

Development Phase: Airbnb Database

1 Create new database PostgreSQL 15

a. Create database sql_mart

```
CREATE DATABASE sql_mart
```

2 Creating table Users and fill it with 20 entries

```
sql_mart=# INSERT INTO Users (user_id, username, email, role) VALUES
sql_mart-# (1, 'user1', 'user1@mailbox.org', 'Host'),
sql_mart-# (2, 'user2', 'user2@mailbox.org', 'Guest'),
sql_mart-# (3, 'user3', 'user3@mailbox.org', 'Admin'),
sql_mart-# (4, 'user4', 'user4@mailbox.org', 'Admin'),
sql_mart-# (5, 'user5', 'user5@mailbox.org', 'Host'),
sql_mart-# (6, 'user6', 'user6@mailbox.org', 'Guest'),
sql_mart-# (7, 'user7', 'user7@mailbox.org', 'Admin'),
sql_mart-# (8, 'user8', 'user8@mailbox.org', 'Host'),
sql_mart-# (9, 'user9', 'user9@mailbox.org', 'Guest'),
sql_mart-# (10, 'user10', 'user10@mailbox.org', 'Admin'),
sql_mart-# (11, 'user11', 'user11@mailbox.org', 'Host'),
sql_mart-# (12, 'user12', 'user12@mailbox.org', 'Guest'),
sql_mart-# (13, 'user13', 'user13@mailbox.org', 'Host'),
sql_mart-# (14, 'user14', 'user14@mailbox.org', 'Host'),
sql_mart-# (15, 'user15', 'user15@mailbox.org', 'Guest'),
sql_mart-# (16, 'user16', 'user16@mailbox.org', 'Admin'),
sql_mart-# (17, 'user17', 'user17@mailbox.org', 'Host'),
sql_mart-# (18, 'user18', 'user18@mailbox.org', 'Guest'),
sql_mart-# (19, 'user19', 'user19@mailbox.org', 'Admin'),
sql_mart-# (20, 'user20', 'user20@mailbox.org', 'Guest'),
sql_mart-# (21, 'user21', 'user21@mailbox.org', 'Admin'),
sql_mart-# (22, 'user22', 'user22@mailbox.org', 'Admin'),
sql_mart-# (23, 'user23', 'user23@mailbox.org', 'Admin'),
sql_mart-# (24, 'user24', 'user24@mailbox.org', 'Admin'),
sql_mart-# (25, 'user25', 'user25@mailbox.org', 'Admin'),
sql_mart-# (26, 'user26', 'user26@mailbox.org', 'Admin'),
sql_mart-# (27, 'user27', 'user27@mailbox.org', 'Admin'),
sql_mart-# (28, 'user28', 'user28@mailbox.org', 'Admin'),
sql_mart-# (29, 'user29', 'user29@mailbox.org', 'Admin'),
sql_mart-# (30, 'user30', 'user30@mailbox.org', 'Admin'),
sql_mart-# (31, 'user31', 'user31@mailbox.org', 'Admin'),
sql_mart-# (32, 'user32', 'user32@mailbox.org', 'Admin'),
sql_mart-# (33, 'user33', 'user33@mailbox.org', 'Admin'),
sql_mart-# (34, 'user34', 'user34@mailbox.org', 'Admin');
INSERT 0 34
sql_mart=# SELECT * FROM Users;
 user_id | username | email | role
-----+-----+-----+-----
 1 | user1 | user1@mailbox.org | Host
 2 | user2 | user2@mailbox.org | Guest
 3 | user3 | user3@mailbox.org | Admin
 4 | user4 | user4@mailbox.org | Admin
 5 | user5 | user5@mailbox.org | Host
 6 | user6 | user6@mailbox.org | Guest
 7 | user7 | user7@mailbox.org | Admin
 8 | user8 | user8@mailbox.org | Host
 9 | user9 | user9@mailbox.org | Guest
10 | user10 | user10@mailbox.org | Admin
11 | user11 | user11@mailbox.org | Host
12 | user12 | user12@mailbox.org | Guest
13 | user13 | user13@mailbox.org | Host
14 | user14 | user14@mailbox.org | Host
15 | user15 | user15@mailbox.org | Guest
16 | user16 | user16@mailbox.org | Admin
17 | user17 | user17@mailbox.org | Host
18 | user18 | user18@mailbox.org | Guest
19 | user19 | user19@mailbox.org | Admin
20 | user20 | user20@mailbox.org | Guest
21 | user21 | user21@mailbox.org | Admin
22 | user22 | user22@mailbox.org | Admin
23 | user23 | user23@mailbox.org | Admin
24 | user24 | user24@mailbox.org | Admin
25 | user25 | user25@mailbox.org | Admin
26 | user26 | user26@mailbox.org | Admin
27 | user27 | user27@mailbox.org | Admin
28 | user28 | user28@mailbox.org | Admin
29 | user29 | user29@mailbox.org | Admin
30 | user30 | user30@mailbox.org | Admin
31 | user31 | user31@mailbox.org | Admin
32 | user32 | user32@mailbox.org | Admin
33 | user33 | user33@mailbox.org | Admin
34 | user34 | user34@mailbox.org | Admin
(34 rows)
```

3 Creating table Employee and fill it with 20 entries

```
sql_mart=# CREATE TABLE Employee (  
sql_mart=#   employee_id INT PRIMARY KEY,  
sql_mart=#   name VARCHAR(50),  
sql_mart=#   manager_id INT,  
sql_mart=#   user_id INT,  
sql_mart=#   FOREIGN KEY (manager_id) REFERENCES Employee(employee_id),  
sql_mart=#   FOREIGN KEY (user_id) REFERENCES Users(user_id)  
sql_mart=# );  
CREATE TABLE  
sql_mart=#  
sql_mart=# INSERT INTO Employee (employee_id, name, manager_id, user_id) VALUES  
sql_mart=#   (1, 'Jessica', NULL, 3),  
sql_mart=#   (2, 'Karen', 1, 4),  
sql_mart=#   (3, 'Tim', 1, 7),  
sql_mart=#   (4, 'Lucas', 1, 10),  
sql_mart=#   (5, 'Thomas', 1, 16),  
sql_mart=#   (6, 'Darl', 1, 19),  
sql_mart=#   (7, 'Sam', 1, 21),  
sql_mart=#   (8, 'Fritz', 1, 22),  
sql_mart=#   (9, 'Howard', 1, 23),  
sql_mart=#   (10, 'Sarah', 1, 24),  
sql_mart=#   (11, 'Aiko', 1, 25),  
sql_mart=#   (12, 'Huesna', 1, 26),  
sql_mart=#   (13, 'Barbara', 1, 27),  
sql_mart=#   (14, 'Kevin', 1, 28),  
sql_mart=#   (15, 'Robert', 1, 29),  
sql_mart=#   (16, 'Stefan', 1, 30),  
sql_mart=#   (17, 'Ramona', 1, 31),  
sql_mart=#   (18, 'Anna', 1, 32),  
sql_mart=#   (19, 'Jeff', 1, 33),  
sql_mart=#   (20, 'Leo', 1, 34);  
INSERT 0 20  
sql_mart=# SELECT * FROM employee ;  
  employee_id |  name  | manager_id | user_id  
-----+-----+-----+-----  
      1 | Jessica |           |      3  
      2 | Karen  |          1 |      4  
      3 | Tim    |          1 |      7  
      4 | Lucas  |          1 |     10  
      5 | Thomas |          1 |     16  
      6 | Darl   |          1 |     19  
      7 | Sam    |          1 |     21  
      8 | Fritz  |          1 |     22  
      9 | Howard |          1 |     23  
     10 | Sarah  |          1 |     24  
     11 | Aiko   |          1 |     25  
     12 | Huesna |          1 |     26  
     13 | Barbara |         1 |     27  
     14 | Kevin  |          1 |     28  
     15 | Robert |          1 |     29  
     16 | Stefan |          1 |     30  
     17 | Ramona |          1 |     31  
     18 | Anna   |          1 |     32  
     19 | Jeff   |          1 |     33  
     20 | Leo    |          1 |     34  
(20 rows)
```


4 Creating table Listings and fill it with 20 entries

```

sql_mart=# CREATE TABLE Listings (
sql_mart=#   listing_id INT PRIMARY KEY,
sql_mart=#   host_id INT,
sql_mart=#   location VARCHAR(100) NOT NULL,
sql_mart=#   description TEXT,
sql_mart=#   price DECIMAL(10,2),
sql_mart=#   FOREIGN KEY (host_id) REFERENCES Users(user_id)
sql_mart=# );
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Listings (listing_id, host_id, location, description, price) VALUES
sql_mart=# (1, 1, 'Paris, France', 'Cozy apartment with a view.', '150.00'),
sql_mart=# (2, 1, 'Barcelona, Spain', 'Sunny beachfront villa with a pool. Ideal for relaxing.', '250.00'),
sql_mart=# (3, 1, 'Rome, Italy', 'Historic loft near the Colosseum.', '180.00'),
sql_mart=# (4, 17, 'Amsterdam, Netherlands', 'Canal-side houseboat with a fireplace.', '300.00'),
sql_mart=# (5, 13, 'Prague, Czech Republic', 'Quaint townhouse in Old Town.', '120.00'),
sql_mart=# (6, 17, 'Dublin, Ireland', 'Modern penthouse with river views.', '400.00'),
sql_mart=# (7, 1, 'Berlin, Germany', 'Artistic loft in a trendy area.', '130.00'),
sql_mart=# (8, 14, 'Athens, Greece', 'Mediterranean villa with sea views.', '280.00'),
sql_mart=# (9, 14, 'Vienna, Austria', 'Elegant townhouse near the city center.', '190.00'),
sql_mart=# (10, 1, 'Edinburgh, Scotland', 'Cozy cottage in the Highlands.', '110.00'),
sql_mart=# (11, 5, 'Budapest, Hungary', 'Riverside apartment with a view.', '170.00'),
sql_mart=# (12, 5, 'Copenhagen, Denmark', 'Scandinavian-style house with a garden.', '220.00'),
sql_mart=# (13, 8, 'Florence, Italy', 'Renaissance villa surrounded by vineyards.', '350.00'),
sql_mart=# (14, 13, 'Krakow, Poland', 'Historic townhouse in Old Town.', '160.00'),
sql_mart=# (15, 17, 'Lisbon, Portugal', 'Sunny apartment with river views.', '500.00'),
sql_mart=# (16, 14, 'Stockholm, Sweden', 'Scenic cabin in the archipelago.', '270.00'),
sql_mart=# (17, 17, 'Dubrovnik, Croatia', 'Seaside villa with a beach.', '140.00'),
sql_mart=# (18, 5, 'Brussels, Belgium', 'Modern loft in the EU district.', '320.00'),
sql_mart=# (19, 8, 'Reykjavik, Iceland', 'Geothermal retreat with Northern Lights views.', '200.00'),
sql_mart=# (20, 1, 'Zurich, Switzerland', 'Alpine chalet with mountain views.', '180.00');
INSERT 0 20
sql_mart=# SELECT * FROM listings ;

```

listing_id	host_id	location	description	price
1	1	Paris, France	Cozy apartment with a view.	150.00
2	1	Barcelona, Spain	Sunny beachfront villa with a pool. Ideal for relaxing.	250.00
3	1	Rome, Italy	Historic loft near the Colosseum.	180.00
4	17	Amsterdam, Netherlands	Canal-side houseboat with a fireplace.	300.00
5	13	Prague, Czech Republic	Quaint townhouse in Old Town.	120.00
6	17	Dublin, Ireland	Modern penthouse with river views.	400.00
7	1	Berlin, Germany	Artistic loft in a trendy area.	130.00
8	14	Athens, Greece	Mediterranean villa with sea views.	280.00
9	14	Vienna, Austria	Elegant townhouse near the city center.	190.00
10	1	Edinburgh, Scotland	Cozy cottage in the Highlands.	110.00
11	5	Budapest, Hungary	Riverside apartment with a view.	170.00
12	5	Copenhagen, Denmark	Scandinavian-style house with a garden.	220.00
13	8	Florence, Italy	Renaissance villa surrounded by vineyards.	350.00
14	13	Krakow, Poland	Historic townhouse in Old Town.	160.00
15	17	Lisbon, Portugal	Sunny apartment with river views.	500.00
16	14	Stockholm, Sweden	Scenic cabin in the archipelago.	270.00
17	17	Dubrovnik, Croatia	Seaside villa with a beach.	140.00
18	5	Brussels, Belgium	Modern loft in the EU district.	320.00
19	8	Reykjavik, Iceland	Geothermal retreat with Northern Lights views.	200.00
20	1	Zurich, Switzerland	Alpine chalet with mountain views.	180.00

(20 rows)

5 Creating table Bookings and fill it with 20 entries

```
sql_mart=# CREATE TABLE Bookings (  
sql_mart=#   booking_id INT PRIMARY KEY,  
sql_mart=#   guest_id INT,  
sql_mart=#   listing_id INT,  
sql_mart=#   dates TIMESTAMP,  
sql_mart=#   payment_status VARCHAR(20),  
sql_mart=#   FOREIGN KEY (guest_id) REFERENCES Users(user_id),  
sql_mart=#   FOREIGN KEY (listing_id) REFERENCES Listings(listing_id)  
sql_mart=# );  
CREATE TABLE  
sql_mart=#  
sql_mart=# INSERT INTO Bookings (booking_id, guest_id, listing_id, dates, payment_status) VALUES  
sql_mart=#   (1, 2, 1, '2022-10-15 12:00:00', 'Paid'),  
sql_mart=#   (2, 6, 1, '2022-11-20 15:00:00', 'Pending'),  
sql_mart=#   (3, 9, 1, '2022-12-25 10:30:00', 'Paid'),  
sql_mart=#   (4, 12, 4, '2023-01-05 08:45:00', 'Paid'),  
sql_mart=#   (5, 15, 5, '2023-02-10 18:20:00', 'Pending'),  
sql_mart=#   (6, 18, 6, '2023-03-15 14:00:00', 'Paid'),  
sql_mart=#   (7, 20, 7, '2023-04-20 09:30:00', 'Paid'),  
sql_mart=#   (8, 2, 8, '2023-05-25 11:45:00', 'Pending'),  
sql_mart=#   (9, 6, 9, '2023-06-30 16:30:00', 'Paid'),  
sql_mart=#   (10, 9, 10, '2023-07-05 13:15:00', 'Paid'),  
sql_mart=#   (11, 12, 11, '2023-08-10 17:00:00', 'Pending'),  
sql_mart=#   (12, 15, 12, '2023-09-15 10:45:00', 'Paid'),  
sql_mart=#   (13, 18, 13, '2023-10-20 14:30:00', 'Paid'),  
sql_mart=#   (14, 20, 14, '2023-11-25 12:00:00', 'Pending'),  
sql_mart=#   (15, 2, 15, '2023-12-30 08:30:00', 'Paid'),  
sql_mart=#   (16, 6, 16, '2024-01-05 16:15:00', 'Paid'),  
sql_mart=#   (17, 9, 17, '2024-02-10 11:00:00', 'Pending'),  
sql_mart=#   (18, 12, 18, '2024-03-15 09:45:00', 'Paid'),  
sql_mart=#   (19, 15, 19, '2024-04-20 13:30:00', 'Paid'),  
sql_mart=#   (20, 18, 20, '2024-05-25 15:00:00', 'Pending');  
INSERT 0 20  
sql_mart=# SELECT * FROM bookings ;  
 booking_id | guest_id | listing_id |      dates      | payment_status  
-----  
      1 |      2 |      1 | 2022-10-15 12:00:00 | Paid  
      2 |      6 |      1 | 2022-11-20 15:00:00 | Pending  
      3 |      9 |      1 | 2022-12-25 10:30:00 | Paid  
      4 |     12 |      4 | 2023-01-05 08:45:00 | Paid  
      5 |     15 |      5 | 2023-02-10 18:20:00 | Pending  
      6 |     18 |      6 | 2023-03-15 14:00:00 | Paid  
      7 |     20 |      7 | 2023-04-20 09:30:00 | Paid  
      8 |      2 |      8 | 2023-05-25 11:45:00 | Pending  
      9 |      6 |      9 | 2023-06-30 16:30:00 | Paid  
     10 |      9 |     10 | 2023-07-05 13:15:00 | Paid  
     11 |     12 |     11 | 2023-08-10 17:00:00 | Pending  
     12 |     15 |     12 | 2023-09-15 10:45:00 | Paid  
     13 |     18 |     13 | 2023-10-20 14:30:00 | Paid  
     14 |     20 |     14 | 2023-11-25 12:00:00 | Pending  
     15 |      2 |     15 | 2023-12-30 08:30:00 | Paid  
     16 |      6 |     16 | 2024-01-05 16:15:00 | Paid  
     17 |      9 |     17 | 2024-02-10 11:00:00 | Pending  
     18 |     12 |     18 | 2024-03-15 09:45:00 | Paid  
     19 |     15 |     19 | 2024-04-20 13:30:00 | Paid  
     20 |     18 |     20 | 2024-05-25 15:00:00 | Pending  
(20 rows)
```

6 Creating table Reviews and fill it with 20 entries

```
sql_mart=# CREATE TABLE Reviews (
sql_mart=#   review_id INT PRIMARY KEY,
sql_mart=#   booking_id INT,
sql_mart=#   user_id INT,
sql_mart=#   listing_id INT,
sql_mart=#   review_text TEXT,
sql_mart=#   rating INT,
sql_mart=#   FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id),
sql_mart=#   FOREIGN KEY (user_id) REFERENCES Users(user_id),
sql_mart=#   FOREIGN KEY (listing_id) REFERENCES Listings(listing_id)
sql_mart=# );
ERROR: relation "reviews" already exists
sql_mart=#
sql_mart=# INSERT INTO Reviews (review_id, booking_id, user_id, listing_id, review_text, rating) VALUES
sql_mart=# (1, 1, 2, 1, 'Absolutely loved the place, felt like a hidden gem!', 5),
sql_mart=# (2, 2, 6, 2, 'Decent spot, but could use a bit more character.', 3),
sql_mart=# (3, 3, 9, 3, 'Incredible stay, exceeded all expectations!', 5),
sql_mart=# (4, 4, 12, 4, 'Breathtaking views, a truly serene experience.', 4),
sql_mart=# (5, 5, 15, 5, 'Average stay, nothing too remarkable.', 3),
sql_mart=# (6, 6, 18, 6, 'Outstanding accommodation, felt like a dream!', 5),
sql_mart=# (7, 7, 20, 7, 'Cozy and inviting, perfect for a relaxing break.', 4),
sql_mart=# (8, 8, 2, 8, 'Bang for the buck, definitely worth it.', 4),
sql_mart=# (9, 9, 6, 9, 'Charming place, every detail was delightful.', 5),
sql_mart=# (10, 10, 9, 10, 'Meh experience, didn't quite hit the mark.', 3),
sql_mart=# (11, 11, 12, 11, 'Quirky and delightful, a stay to remember!', 5),
sql_mart=# (12, 12, 15, 12, 'Spotless property, cleanliness was top-notch.', 4),
sql_mart=# (13, 13, 18, 13, 'Exceptional service, made me feel right at home.', 5),
sql_mart=# (14, 14, 20, 14, 'Fair stay, had its ups and downs.', 3),
sql_mart=# (15, 15, 2, 15, 'Luxurious vibes, felt like royalty!', 5),
sql_mart=# (16, 16, 6, 16, 'Tranquil retreat, perfect for unwinding.', 4),
sql_mart=# (17, 17, 9, 17, 'Good location, but amenities could be better.', 3),
sql_mart=# (18, 18, 12, 18, 'Top-notch stay, highly recommend to all!', 5),
sql_mart=# (19, 19, 15, 19, 'Quaint and charming, a true hidden treasure.', 4),
sql_mart=# (20, 20, 18, 20, 'Comfortable stay, but lacked that special touch.', 3);
INSERT 0 20
sql_mart=# SELECT * FROM reviews ;
 review_id | booking_id | user_id | listing_id |          review_text          | rating
-----+-----+-----+-----+-----+-----
1 | 1 | 2 | 1 | Absolutely loved the place, felt like a hidden gem! | 5
2 | 2 | 6 | 2 | Decent spot, but could use a bit more character. | 3
3 | 3 | 9 | 3 | Incredible stay, exceeded all expectations! | 5
4 | 4 | 12 | 4 | Breathtaking views, a truly serene experience. | 4
5 | 5 | 15 | 5 | Average stay, nothing too remarkable. | 3
6 | 6 | 18 | 6 | Outstanding accommodation, felt like a dream! | 5
7 | 7 | 20 | 7 | Cozy and inviting, perfect for a relaxing break. | 4
8 | 8 | 2 | 8 | Bang for the buck, definitely worth it. | 4
9 | 9 | 6 | 9 | Charming place, every detail was delightful. | 5
10 | 10 | 9 | 10 | Meh experience, didn't quite hit the mark. | 3
11 | 11 | 12 | 11 | Quirky and delightful, a stay to remember! | 5
12 | 12 | 15 | 12 | Spotless property, cleanliness was top-notch. | 4
13 | 13 | 18 | 13 | Exceptional service, made me feel right at home. | 5
14 | 14 | 20 | 14 | Fair stay, had its ups and downs. | 3
15 | 15 | 2 | 15 | Luxurious vibes, felt like royalty! | 5
16 | 16 | 6 | 16 | Tranquil retreat, perfect for unwinding. | 4
17 | 17 | 9 | 17 | Good location, but amenities could be better. | 3
18 | 18 | 12 | 18 | Top-notch stay, highly recommend to all! | 5
19 | 19 | 15 | 19 | Quaint and charming, a true hidden treasure. | 4
20 | 20 | 18 | 20 | Comfortable stay, but lacked that special touch. | 3
(20 rows)
```


7 Creating table Availability_Calendar and fill it with 20 entries

```
sql_mart=# CREATE TABLE Availability_Calendar (
sql_mart(#   listing_id INT,
sql_mart(#   date TIMESTAMP,
sql_mart(#   availability_status VARCHAR(20),
sql_mart(#   PRIMARY KEY (listing_id, date),
sql_mart(#   FOREIGN KEY (listing_id) REFERENCES Listings(listing_id)
sql_mart(# );
CREATE TABLE
sql_mart=# INSERT INTO Availability_Calendar (listing_id, date, availability_status)
sql_mart=# VALUES
sql_mart=# (1, '2024-10-15 08:30:00', 'Available'), -- listing_id 1
sql_mart=# (1, '2024-10-16 10:45:00', 'Booked'),-- listing_id 1
sql_mart=# (1, '2024-10-17 12:15:00', 'Available'),-- listing_id 1
sql_mart=# (4, '2024-10-18 14:30:00', 'Available'),
sql_mart=# (5, '2024-10-19 16:45:00', 'Booked'),
sql_mart=# (6, '2024-10-20 09:00:00', 'Available'),
sql_mart=# (7, '2024-10-21 11:30:00', 'Available'),
sql_mart=# (8, '2024-10-22 13:45:00', 'Booked'),
sql_mart=# (9, '2024-10-23 15:00:00', 'Available'),
sql_mart=# (10, '2024-10-24 17:15:00', 'Available'),
sql_mart=# (11, '2024-10-25 19:30:00', 'Booked'),
sql_mart=# (12, '2024-10-26 08:45:00', 'Available'),
sql_mart=# (13, '2024-10-27 10:00:00', 'Available'),
sql_mart=# (14, '2024-10-28 12:30:00', 'Booked'),
sql_mart=# (15, '2024-10-29 14:45:00', 'Available'),
sql_mart=# (16, '2024-10-30 17:00:00', 'Available'),
sql_mart=# (17, '2024-10-31 19:15:00', 'Booked'),
sql_mart=# (18, '2024-11-01 08:30:00', 'Available'),
sql_mart=# (19, '2024-11-02 10:45:00', 'Available'),
sql_mart=# (20, '2024-11-03 12:00:00', 'Booked');
INSERT 0 20
sql_mart=# SELECT * FROM availability_calendar ;
 listing_id |          date          | availability_status
-----+-----+-----
      1 | 2024-10-15 08:30:00 | Available
      1 | 2024-10-16 10:45:00 | Booked
      1 | 2024-10-17 12:15:00 | Available
      4 | 2024-10-18 14:30:00 | Available
      5 | 2024-10-19 16:45:00 | Booked
      6 | 2024-10-20 09:00:00 | Available
      7 | 2024-10-21 11:30:00 | Available
      8 | 2024-10-22 13:45:00 | Booked
      9 | 2024-10-23 15:00:00 | Available
     10 | 2024-10-24 17:15:00 | Available
     11 | 2024-10-25 19:30:00 | Booked
     12 | 2024-10-26 08:45:00 | Available
     13 | 2024-10-27 10:00:00 | Available
     14 | 2024-10-28 12:30:00 | Booked
     15 | 2024-10-29 14:45:00 | Available
     16 | 2024-10-30 17:00:00 | Available
     17 | 2024-10-31 19:15:00 | Booked
     18 | 2024-11-01 08:30:00 | Available
     19 | 2024-11-02 10:45:00 | Available
     20 | 2024-11-03 12:00:00 | Booked
(20 rows)
```

8 Creating table Platform_Policies and fill it with 20 entries

```
sql_mart=# CREATE TABLE Platform_Policies (  
sql_mart=#   policy_id INT PRIMARY KEY,  
sql_mart=#   policy_description TEXT  
sql_mart=# );  
CREATE TABLE  
sql_mart=#  
sql_mart=# INSERT INTO Platform_Policies (policy_id, policy_description) VALUES  
sql_mart=#   (1, 'Privacy and security first! Your data is safe with us.'),  
sql_mart=#   (2, 'Top-notch customer service is our priority.'),  
sql_mart=#   (3, 'Transparency matters. Stay informed every step.'),  
sql_mart=#   (4, 'Diversity and inclusivity are core values.'),  
sql_mart=#   (5, 'Safety is paramount. Your security is our focus.'),  
sql_mart=#   (6, 'Your feedback is valued. Open communication is key.'),  
sql_mart=#   (7, 'Respect and professionalism always.'),  
sql_mart=#   (8, 'Supporting sustainability for a better future.'),  
sql_mart=#   (9, 'Continuous improvement drives us forward.'),  
sql_mart=#   (10, 'Honesty and integrity define our community.'),  
sql_mart=#   (11, 'Innovation and creativity are encouraged.'),  
sql_mart=#   (12, 'Positivity and support make our community thrive.'),  
sql_mart=#   (13, 'Fairness and equality for all.'),  
sql_mart=#   (14, 'High ethical standards guide us.'),  
sql_mart=#   (15, 'Empowerment and autonomy for users.'),  
sql_mart=#   (16, 'User satisfaction is our priority.'),  
sql_mart=#   (17, 'Collaboration leads to success.'),  
sql_mart=#   (18, 'A culture of learning and growth.'),  
sql_mart=#   (19, 'Adaptability and flexibility are valued.'),  
sql_mart=#   (20, 'Making a positive impact together.');  
INSERT 0 20  
sql_mart=# SELECT * FROM platform_policies ;  
policy_id | policy_description  
-----+-----  
1 | Privacy and security first! Your data is safe with us.  
2 | Top-notch customer service is our priority.  
3 | Transparency matters. Stay informed every step.  
4 | Diversity and inclusivity are core values.  
5 | Safety is paramount. Your security is our focus.  
6 | Your feedback is valued. Open communication is key.  
7 | Respect and professionalism always.  
8 | Supporting sustainability for a better future.  
9 | Continuous improvement drives us forward.  
10 | Honesty and integrity define our community.  
11 | Innovation and creativity are encouraged.  
12 | Positivity and support make our community thrive.  
13 | Fairness and equality for all.  
14 | High ethical standards guide us.  
15 | Empowerment and autonomy for users.  
16 | User satisfaction is our priority.  
17 | Collaboration leads to success.  
18 | A culture of learning and growth.  
19 | Adaptability and flexibility are valued.  
20 | Making a positive impact together.  
(20 rows)
```


9 Creating table Super_Host_Status and fill it with 20 entries

```
sql_mart=# CREATE TABLE Super_Host_Status (  
sql_mart=#   host_id INT,  
sql_mart=#   super_host_status BOOL,  
sql_mart=#   PRIMARY KEY (host_id),  
sql_mart=#   FOREIGN KEY (host_id) REFERENCES Users(user_id)  
sql_mart=# );  
CREATE TABLE  
sql_mart=#  
sql_mart=# INSERT INTO Super_Host_Status (host_id, super_host_status) VALUES  
sql_mart=# (1, TRUE),  
sql_mart=# (5, TRUE),  
sql_mart=# (8, TRUE),  
sql_mart=# (11, TRUE),  
sql_mart=# (13, TRUE),  
sql_mart=# (14, TRUE),  
sql_mart=# (17, TRUE),  
sql_mart=# (2, FALSE),  
sql_mart=# (3, FALSE),  
sql_mart=# (4, FALSE),  
sql_mart=# (6, FALSE),  
sql_mart=# (7, FALSE),  
sql_mart=# (9, FALSE),  
sql_mart=# (10, FALSE),  
sql_mart=# (18, FALSE),  
sql_mart=# (19, FALSE),  
sql_mart=# (20, FALSE),  
sql_mart=# (21, FALSE),  
sql_mart=# (22, FALSE),  
sql_mart=# (23, FALSE);  
INSERT 0 20  
sql_mart=# SELECT * FROM super_host_status ;  
      host_id | super_host_status  
-----+-----  
          1 | t  
          5 | t  
          8 | t  
         11 | t  
         13 | t  
         14 | t  
         17 | t  
          2 | f  
          3 | f  
          4 | f  
          6 | f  
          7 | f  
          9 | f  
         10 | f  
         18 | f  
         19 | f  
         20 | f  
         21 | f  
         22 | f  
         23 | f  
(20 rows)
```

10 Creating table User_Reviews and fill it with 20 entries

```
sql_mart=# CREATE TABLE User_Reviews (  
sql_mart(#   user_id INT,  
sql_mart(#   review_id INT,  
sql_mart(#   PRIMARY KEY (user_id, review_id),  
sql_mart(#   FOREIGN KEY (user_id) REFERENCES Users(user_id),  
sql_mart(#   FOREIGN KEY (review_id) REFERENCES Reviews(review_id)  
sql_mart(# );  
CREATE TABLE  
sql_mart=#  
sql_mart=# INSERT INTO User_Reviews (user_id, review_id) VALUES  
sql_mart-#      (2, 1),  
sql_mart-#      (6, 2),  
sql_mart-#      (9, 3),  
sql_mart-#      (12, 4),  
sql_mart-#      (15, 5),  
sql_mart-#      (18, 6),  
sql_mart-#      (20, 7),  
sql_mart-#      (2, 8),  
sql_mart-#      (6, 9),  
sql_mart-#      (9, 10),  
sql_mart-#      (12, 11),  
sql_mart-#      (15, 12),  
sql_mart-#      (18, 13),  
sql_mart-#      (20, 14),  
sql_mart-#      (2, 15),  
sql_mart-#      (6, 16),  
sql_mart-#      (9, 17),  
sql_mart-#      (12, 18),  
sql_mart-#      (15, 19),  
sql_mart-#      (18, 20);  
INSERT 0 20  
sql_mart=# SELECT * FROM user_reviews ;  
user_id | review_id  
-----+-----  
2 | 1  
6 | 2  
9 | 3  
12 | 4  
15 | 5  
18 | 6  
20 | 7  
2 | 8  
6 | 9  
9 | 10  
12 | 11  
15 | 12  
18 | 13  
20 | 14  
2 | 15  
6 | 16  
9 | 17  
12 | 18  
15 | 19  
18 | 20  
(20 rows)
```


11 Creating table User_Policies and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO User_Policies (user_id, policy_id) VALUES
sql_mart-#      (1, 1),
sql_mart-#      (1, 2),
sql_mart-#      (1, 3),
sql_mart-#      (2, 4),
sql_mart-#      (3, 5),
sql_mart-#      (4, 6),
sql_mart-#      (5, 7),
sql_mart-#      (6, 8),
sql_mart-#      (7, 9),
sql_mart-#      (8, 10),
sql_mart-#      (9, 11),
sql_mart-#      (10, 12),
sql_mart-#      (11, 1),
sql_mart-#      (12, 14),
sql_mart-#      (13, 15),
sql_mart-#      (14, 16),
sql_mart-#      (15, 17),
sql_mart-#      (16, 18),
sql_mart-#      (17, 19),
sql_mart-#      (18, 20);
INSERT 0 20
sql_mart=# SELECT * FROM user_policies ;
 user_id | policy_id
-----+-----
      1 |      1
      1 |      2
      1 |      3
      2 |      4
      3 |      5
      4 |      6
      5 |      7
      6 |      8
      7 |      9
      8 |     10
      9 |     11
     10 |     12
     11 |      1
     12 |     14
     13 |     15
     14 |     16
     15 |     17
     16 |     18
     17 |     19
     18 |     20
(20 rows)
```

12 Creating table Cancellation_Policy and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Cancellation_Policy (policy_id, listing_id, policy_description) VALUES
sql_mart=# (1, 1, 'Guests can cancel up to 7 days before check-in for a full refund. '),
sql_mart=# (2, 1, 'Guests can cancel up to 3 days before check-in for a full refund. '),
sql_mart=# (3, 3, 'Guests can cancel up to 14 days before check-in for a full refund. '),
sql_mart=# (4, 4, 'Guests can cancel up to 30 days before check-in for a full refund. '),
sql_mart=# (5, 5, 'Guests can cancel up to 1 day before check-in for a full refund. '),
sql_mart=# (6, 6, 'Guests can cancel up to 2 days before check-in for a full refund. '),
sql_mart=# (7, 7, 'Guests can cancel up to 14 days before check-in for a 50% refund. '),
sql_mart=# (8, 8, 'Guests can cancel up to 30 days before check-in for a 75% refund. '),
sql_mart=# (9, 9, 'Guests can cancel up to 7 days before check-in for a 50% refund. '),
sql_mart=# (10, 10, 'Guests can cancel up to 3 days before check-in for a 75% refund. '),
sql_mart=# (11, 11, 'Guests can cancel up to 14 days before check-in for a 75% refund. '),
sql_mart=# (12, 12, 'Guests can cancel up to 30 days before check-in for a 50% refund. '),
sql_mart=# (13, 13, 'No refunds will be given for cancellations. '),
sql_mart=# (14, 14, 'Guests can reschedule their stay for free within 1 year of the original check-in date. '),
sql_mart=# (15, 15, 'Guests can cancel up to 14 days before check-in for a full refund, but a 5% processing fee will apply. '),
sql_mart=# (16, 16, 'Guests can cancel up to 30 days before check-in for a full refund, but a 10% processing fee will apply. '),
sql_mart=# (17, 17, 'Guests can cancel up to 7 days before check-in for a 50% refund, but a 5% processing fee will apply. '),
sql_mart=# (18, 18, 'Guests can cancel up to 3 days before check-in for a 75% refund, but a 10% processing fee will apply. '),
sql_mart=# (19, 19, 'Guests can cancel up to 14 days before check-in for a 75% refund, but a 5% processing fee will apply. '),
sql_mart=# (20, 20, 'Guests can cancel up to 30 days before check-in for a 50% refund, but a 10% processing fee will apply. ');
INSERT 0 20
sql_mart=# SELECT * FROM cancellation_policy ;
 policy_id | listing_id | policy_description
-----
1 | 1 | Guests can cancel up to 7 days before check-in for a full refund.
2 | 1 | Guests can cancel up to 3 days before check-in for a full refund.
3 | 3 | Guests can cancel up to 14 days before check-in for a full refund.
4 | 4 | Guests can cancel up to 30 days before check-in for a full refund.
5 | 5 | Guests can cancel up to 1 day before check-in for a full refund.
6 | 6 | Guests can cancel up to 2 days before check-in for a full refund.
7 | 7 | Guests can cancel up to 14 days before check-in for a 50% refund.
8 | 8 | Guests can cancel up to 30 days before check-in for a 75% refund.
9 | 9 | Guests can cancel up to 7 days before check-in for a 50% refund.
10 | 10 | Guests can cancel up to 3 days before check-in for a 75% refund.
11 | 11 | Guests can cancel up to 14 days before check-in for a 75% refund.
12 | 12 | Guests can cancel up to 30 days before check-in for a 50% refund.
13 | 13 | No refunds will be given for cancellations.
14 | 14 | Guests can reschedule their stay for free within 1 year of the original check-in date.
15 | 15 | Guests can cancel up to 14 days before check-in for a full refund, but a 5% processing fee will apply.
16 | 16 | Guests can cancel up to 30 days before check-in for a full refund, but a 10% processing fee will apply.
17 | 17 | Guests can cancel up to 7 days before check-in for a 50% refund, but a 5% processing fee will apply.
18 | 18 | Guests can cancel up to 3 days before check-in for a 75% refund, but a 10% processing fee will apply.
19 | 19 | Guests can cancel up to 14 days before check-in for a 75% refund, but a 5% processing fee will apply.
20 | 20 | Guests can cancel up to 30 days before check-in for a 50% refund, but a 10% processing fee will apply.
(20 rows)
```

13 Creating table Wish_List and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Wish_List (wish_id, guest_id, listing_id) VALUES
sql_mart-# (1, 2, 1),
sql_mart-# (2, 6, 2),
sql_mart-# (3, 9, 3),
sql_mart-# (4, 12, 4),
sql_mart-# (5, 15, 5),
sql_mart-# (6, 18, 6),
sql_mart-# (7, 20, 7),
sql_mart-# (8, 2, 8),
sql_mart-# (9, 6, 9),
sql_mart-# (10, 9, 10),
sql_mart-# (11, 12, 11),
sql_mart-# (12, 15, 12),
sql_mart-# (13, 18, 13),
sql_mart-# (14, 20, 14),
sql_mart-# (15, 2, 15),
sql_mart-# (16, 6, 16),
sql_mart-# (17, 9, 17),
sql_mart-# (18, 12, 18),
sql_mart-# (19, 15, 19),
sql_mart-# (20, 18, 20);
INSERT 0 20
sql_mart=#
sql_mart=# SELECT * FROM wish_list ;
 wish_id | guest_id | listing_id
-----+-----+-----
      1 |         2 |         1
      2 |         6 |         2
      3 |         9 |         3
      4 |        12 |         4
      5 |        15 |         5
      6 |        18 |         6
      7 |        20 |         7
      8 |         2 |         8
      9 |         6 |         9
     10 |         9 |        10
     11 |        12 |        11
     12 |        15 |        12
     13 |        18 |        13
     14 |        20 |        14
     15 |         2 |        15
     16 |         6 |        16
     17 |         9 |        17
     18 |        12 |        18
     19 |        15 |        19
     20 |        18 |        20
(20 rows)
```


14 Creating table Amenities and fill it with 20 entries

```
sql_mart=# CREATE TABLE Amenities (  
sql_mart=#   amenity_id INT PRIMARY KEY,  
sql_mart=#   listing_id INT,  
sql_mart=#   amenity_description TEXT,  
sql_mart=#   FOREIGN KEY (listing_id) REFERENCES Listings(listing_id)  
sql_mart=# );  
CREATE TABLE  
sql_mart=#  
sql_mart=# INSERT INTO Amenities (amenity_id, listing_id, amenity_description) VALUES  
sql_mart=# (1, 1, 'Fully equipped kitchen with modern appliances'),  
sql_mart=# (2, 1, 'Spacious living room with a comfortable sofa'),  
sql_mart=# (3, 3, 'Private balcony with city views'),  
sql_mart=# (4, 4, 'Dining area with seating for six'),  
sql_mart=# (5, 5, 'Free parking on the premises'),  
sql_mart=# (6, 6, 'High-speed Wi-Fi and a dedicated workspace'),  
sql_mart=# (7, 7, 'Smart TV with streaming services'),  
sql_mart=# (8, 8, 'Washer and dryer in the unit'),  
sql_mart=# (9, 9, 'Air conditioning and heating'),  
sql_mart=# (10, 10, 'Iron and ironing board'),  
sql_mart=# (11, 11, 'Hair dryer'),  
sql_mart=# (12, 12, 'Coffee maker and kettle'),  
sql_mart=# (13, 13, 'Shampoo, conditioner, and body wash'),  
sql_mart=# (14, 14, 'Bathrobes and slippers'),  
sql_mart=# (15, 15, 'Free bottled water and snacks'),  
sql_mart=# (16, 16, 'Gym access'),  
sql_mart=# (17, 17, 'Pool access'),  
sql_mart=# (18, 18, 'BBQ facilities'),  
sql_mart=# (19, 19, 'Elevator access'),  
sql_mart=# (20, 20, '24/7 security surveillance');  
INSERT 0 20  
sql_mart=# SELECT * FROM amenities ;  
   amenity_id | listing_id |          amenity_description  
-----+-----+-----  
      1 |      1 | Fully equipped kitchen with modern appliances  
      2 |      1 | Spacious living room with a comfortable sofa  
      3 |      3 | Private balcony with city views  
      4 |      4 | Dining area with seating for six  
      5 |      5 | Free parking on the premises  
      6 |      6 | High-speed Wi-Fi and a dedicated workspace  
      7 |      7 | Smart TV with streaming services  
      8 |      8 | Washer and dryer in the unit  
      9 |      9 | Air conditioning and heating  
     10 |     10 | Iron and ironing board  
     11 |     11 | Hair dryer  
     12 |     12 | Coffee maker and kettle  
     13 |     13 | Shampoo, conditioner, and body wash  
     14 |     14 | Bathrobes and slippers  
     15 |     15 | Free bottled water and snacks  
     16 |     16 | Gym access  
     17 |     17 | Pool access  
     18 |     18 | BBQ facilities  
     19 |     19 | Elevator access  
     20 |     20 | 24/7 security surveillance  
(20 rows)
```


15 Creating table House_Rules and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO House_Rules (rule_id, listing_id, rule_description) VALUES
sql_mart-# (1, 1, 'No smoking.'),
sql_mart-# (2, 1, 'No alcohol.'),
sql_mart-# (3, 3, 'No more than 2 people.'),
sql_mart-# (4, 4, 'Must be cleaned daily.'),
sql_mart-# (5, 5, 'No loud music.'),
sql_mart-# (6, 6, 'Please turn off heater when leaving the rooms.'),
sql_mart-# (7, 7, 'No additional visitors allowed.'),
sql_mart-# (8, 8, 'No pets allowed.'),
sql_mart-# (9, 9, 'Leave the bathroom door open after use.'),
sql_mart-# (10, 10, 'Open windows daily.'),
sql_mart-# (11, 11, 'No shoes allowed indoor.'),
sql_mart-# (12, 12, 'Door must be locked at night.'),
sql_mart-# (13, 13, 'Must return before 11 pm.'),
sql_mart-# (14, 14, 'Quiet hours are from 10 PM to 8 AM'),
sql_mart-# (15, 15, 'Please do not smoke inside the house. Smoking is only allowed in designated outdoor areas.'),
sql_mart-# (16, 16, 'Please remove your shoes upon entering the house to keep the floors clean.'),
sql_mart-# (17, 17, 'Please do not move any furniture without prior permission from the host'),
sql_mart-# (18, 18, 'Quiet hours are from 11 PM to 6 AM'),
sql_mart-# (19, 19, 'Guests are responsible for washing their dishes and putting them away before checking out.'),
sql_mart-# (20, 20, 'Please turn off all lights, appliances, and electronics when not in use to conserve energy');
INSERT 0 20
sql_mart=# SELECT * FROM house_rules ;
 rule_id | listing_id | rule_description
-----+-----+-----
 1 | 1 | No smoking.
 2 | 1 | No alcohol.
 3 | 3 | No more than 2 people.
 4 | 4 | Must be cleaned daily.
 5 | 5 | No loud music.
 6 | 6 | Please turn off heater when leaving the rooms.
 7 | 7 | No additional visitors allowed.
 8 | 8 | No pets allowed.
 9 | 9 | Leave the bathroom door open after use.
10 | 10 | Open windows daily.
11 | 11 | No shoes allowed indoor.
12 | 12 | Door must be locked at night.
13 | 13 | Must return before 11 pm.
14 | 14 | Quiet hours are from 10 PM to 8 AM
15 | 15 | Please do not smoke inside the house. Smoking is only allowed in designated outdoor areas.
16 | 16 | Please remove your shoes upon entering the house to keep the floors clean.
17 | 17 | Please do not move any furniture without prior permission from the host
18 | 18 | Quiet hours are from 11 PM to 6 AM
19 | 19 | Guests are responsible for washing their dishes and putting them away before checking out.
20 | 20 | Please turn off all lights, appliances, and electronics when not in use to conserve energy
(20 rows)
```

16 Creating table Neighborhood and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Neighborhood (neighborhood_id, listing_id, neighborhood_description) VALUES
sql_mart-# (1, 1, 'Downtown, close to popular tourist attractions and nightlife.'),
sql_mart-# (2, 2, 'Quiet residential neighborhood with easy access to public transportation.'),
sql_mart-# (3, 3, 'Beachfront community with water sports and outdoor activities.'),
sql_mart-# (4, 4, 'Historic district with museums, galleries, and cultural landmarks.'),
sql_mart-# (5, 5, 'Upscale shopping and dining district with designer boutiques and fine dining.'),
sql_mart-# (6, 6, 'Artsy neighborhood with street art, galleries, and theaters.'),
sql_mart-# (7, 7, 'Family-friendly neighborhood with parks, playgrounds, and family-oriented activities.'),
sql_mart-# (8, 8, 'Business district with high-rise offices and hotels.'),
sql_mart-# (9, 9, 'Waterfront neighborhood with marinas, boat tours, and seafood restaurants.'),
sql_mart-# (10, 10, 'Cultural district with music venues, theaters, and art installations.'),
sql_mart-# (11, 11, 'Mountain resort community with skiing, snowboarding, and hiking trails.'),
sql_mart-# (12, 12, 'College town with bars, cafes, and a vibrant student population.'),
sql_mart-# (13, 13, 'Industrial district with trendy lofts, breweries, and art studios.'),
sql_mart-# (14, 14, 'International district with diverse cuisine, markets, and cultural experiences.'),
sql_mart-# (15, 15, 'Seaside community with beaches, fishing, and seafood.'),
sql_mart-# (16, 16, 'Lakefront community with water sports, boating, and outdoor recreation.'),
sql_mart-# (17, 17, 'Rural community with farms, orchards, and outdoor activities.'),
sql_mart-# (18, 18, 'Desert community with hiking, biking, and natural wonders.'),
sql_mart-# (19, 19, 'Coastal community with beaches, surfing, and marine life.'),
sql_mart-# (20, 20, 'Mountain resort with water sports, and outdoor activities.');
```

```
INSERT 0 20
sql_mart=# SELECT * FROM neighborhood ;
 neighborhood_id | listing_id | neighborhood_description
-----
1 | 1 | Downtown, close to popular tourist attractions and nightlife.
2 | 2 | Quiet residential neighborhood with easy access to public transportation.
3 | 3 | Beachfront community with water sports and outdoor activities.
4 | 4 | Historic district with museums, galleries, and cultural landmarks.
5 | 5 | Upscale shopping and dining district with designer boutiques and fine dining.
6 | 6 | Artsy neighborhood with street art, galleries, and theaters.
7 | 7 | Family-friendly neighborhood with parks, playgrounds, and family-oriented activities.
8 | 8 | Business district with high-rise offices and hotels.
9 | 9 | Waterfront neighborhood with marinas, boat tours, and seafood restaurants.
10 | 10 | Cultural district with music venues, theaters, and art installations.
11 | 11 | Mountain resort community with skiing, snowboarding, and hiking trails.
12 | 12 | College town with bars, cafes, and a vibrant student population.
13 | 13 | Industrial district with trendy lofts, breweries, and art studios.
14 | 14 | International district with diverse cuisine, markets, and cultural experiences.
15 | 15 | Seaside community with beaches, fishing, and seafood.
16 | 16 | Lakefront community with water sports, boating, and outdoor recreation.
17 | 17 | Rural community with farms, orchards, and outdoor activities.
18 | 18 | Desert community with hiking, biking, and natural wonders.
19 | 19 | Coastal community with beaches, surfing, and marine life.
20 | 20 | Mountain resort with water sports, and outdoor activities.
```

(20 rows)

17 Creating table Location and fill it with 20 entities

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Location (location_id, listing_id, address, city, state, country, zip_code) VALUES
sql_mart=# (1, 1, 'Address 1', 'Paris', 'State 1', 'France', 'Zip 1'),
sql_mart=# (2, 2, 'Address 2', 'Barcelona', 'State 2', 'Spain', 'Zip 2'),
sql_mart=# (3, 3, 'Address 3', 'Rome', 'State 3', 'Italy', 'Zip 3'),
sql_mart=# (4, 4, 'Address 4', 'Amsterdam', 'State 4', 'Netherlands', 'Zip 4'),
sql_mart=# (5, 5, 'Address 5', 'Prague', 'State 5', 'Czech Republic', 'Zip 5'),
sql_mart=# (6, 6, 'Address 6', 'Dublin', 'State 6', 'Ireland', 'Zip 6'),
sql_mart=# (7, 7, 'Address 7', 'Berlin', 'State 7', 'Germany', 'Zip 7'),
sql_mart=# (8, 8, 'Address 8', 'Athens', 'State 8', 'Greece', 'Zip 8'),
sql_mart=# (9, 9, 'Address 9', 'Vienna', 'State 9', 'Austria', 'Zip 9'),
sql_mart=# (10, 10, 'Address 10', 'Edinburgh', 'State 10', 'Scotland', 'Zip 10'),
sql_mart=# (11, 11, 'Address 11', 'Budapest', 'State 11', 'Hungary', 'Zip 11'),
sql_mart=# (12, 12, 'Address 12', 'Copenhagen', 'State 12', 'Denmark', 'Zip 12'),
sql_mart=# (13, 13, 'Address 13', 'Florence', 'State 13', 'Italy', 'Zip 13'),
sql_mart=# (14, 14, 'Address 14', 'Krakow', 'State 14', 'Poland', 'Zip 14'),
sql_mart=# (15, 15, 'Address 15', 'Lisbon', 'State 15', 'Portugal', 'Zip 15'),
sql_mart=# (16, 16, 'Address 16', 'Stockholm', 'State 16', 'Sweden', 'Zip 16'),
sql_mart=# (17, 17, 'Address 17', 'Dubrovnik', 'State 17', 'Croatia', 'Zip 17'),
sql_mart=# (18, 18, 'Address 18', 'Brussels', 'State 18', 'Belgium', 'Zip 18'),
sql_mart=# (19, 19, 'Address 19', 'Reykjavik', 'State 19', 'Iceland', 'Zip 19'),
sql_mart=# (20, 20, 'Address 20', 'Zurich', 'State 20', 'Switzerland', 'Zip 20');
INSERT 0 20
sql_mart=# SELECT * FROM location ;
```

location_id	listing_id	address	city	state	country	zip_code
1	1	Address 1	Paris	State 1	France	Zip 1
2	2	Address 2	Barcelona	State 2	Spain	Zip 2
3	3	Address 3	Rome	State 3	Italy	Zip 3
4	4	Address 4	Amsterdam	State 4	Netherlands	Zip 4
5	5	Address 5	Prague	State 5	Czech Republic	Zip 5
6	6	Address 6	Dublin	State 6	Ireland	Zip 6
7	7	Address 7	Berlin	State 7	Germany	Zip 7
8	8	Address 8	Athens	State 8	Greece	Zip 8
9	9	Address 9	Vienna	State 9	Austria	Zip 9
10	10	Address 10	Edinburgh	State 10	Scotland	Zip 10
11	11	Address 11	Budapest	State 11	Hungary	Zip 11
12	12	Address 12	Copenhagen	State 12	Denmark	Zip 12
13	13	Address 13	Florence	State 13	Italy	Zip 13
14	14	Address 14	Krakow	State 14	Poland	Zip 14
15	15	Address 15	Lisbon	State 15	Portugal	Zip 15
16	16	Address 16	Stockholm	State 16	Sweden	Zip 16
17	17	Address 17	Dubrovnik	State 17	Croatia	Zip 17
18	18	Address 18	Brussels	State 18	Belgium	Zip 18
19	19	Address 19	Reykjavik	State 19	Iceland	Zip 19
20	20	Address 20	Zurich	State 20	Switzerland	Zip 20

(20 rows)

18 Creating table Messages and fill it with 20 entries

```
CREATE TABLE
sql>CREATE TABLE Messages (message_id, guest_id, host_id, listing_id, message_text, timestamp) VALUES
sql>INSERT INTO Messages (message_id, guest_id, host_id, listing_id, message_text, timestamp) VALUES
sql>-- (1, 2, 1, 1, 'Hi, I just checked in. The place looks great!', '2024-01-01 00:00:00'),
sql>-- (2, 6, 1, 2, 'Hi, thanks for staying with me! I am glad you like it. Let me know if you need anything.', '2024-01-02 00:00:00'),
sql>-- (3, 9, 1, 3, 'Hi, I was wondering if you could tell me where the nearest grocery store is?', '2024-01-03 00:00:00'),
sql>-- (4, 12, 17, 4, 'Hi, sure thing! There is a grocery store about a mile away on Main Street.', '2024-01-04 00:00:00'),
sql>-- (5, 15, 13, 5, 'Hi, I also wanted to let you know that I will be checking out a little early tomorrow.', '2024-01-05 00:00:00'),
sql>-- (6, 18, 17, 6, 'Hi, no problem. Just let me know if you need any help with that.', '2024-01-06 00:00:00'),
sql>-- (7, 20, 1, 7, 'Hi, I was wondering if it is okay if I have a guest over tonight?', '2024-01-07 00:00:00'),
sql>-- (8, 2, 14, 8, 'Hi, sure, just let me know who it is and what time they will be arriving.', '2024-01-08 00:00:00'),
sql>-- (9, 6, 14, 9, 'Hi, I am also curious if there is a good place to get breakfast around here?', '2024-01-09 00:00:00'),
sql>-- (10, 9, 1, 10, 'Hi, yes, there is a great little cafe called The Daily Grind about a block away.', '2024-01-10 00:00:00'),
sql>-- (11, 12, 5, 11, 'Hi, I just wanted to let you know that I am checking out now.', '2024-01-11 00:00:00'),
sql>-- (12, 15, 5, 12, 'Hi, thanks for staying here! I hope you had a great stay. Let me know if you have any feedback.', '2024-01-12 00:00:00'),
sql>-- (13, 18, 8, 13, 'Hi, I will leave a review for you soon. I really enjoyed my stay!', '2024-01-13 00:00:00'),
sql>-- (14, 20, 13, 14, 'Hi, thank you! I am glad you had a good stay. I will look forward to your review.', '2024-01-14 00:00:00'),
sql>-- (15, 2, 17, 15, 'Hi, I just wanted to let you know that I had a great time and I will definitely be back!', '2024-01-15 00:00:00'),
sql>-- (16, 6, 14, 16, 'Hi, thank you! I am glad to hear that. I hope to see you again soon!', '2024-01-16 00:00:00'),
sql>-- (17, 9, 17, 17, 'Hi, I also wanted to let you know that I left a little something for you as a thank you.', '2024-01-17 00:00:00'),
sql>-- (18, 12, 5, 18, 'Hi, I was wondering if you could tell me where the nearest pharmacy is? I need to pick up some medication.', '2024-01-18 00:00:00'),
sql>-- (19, 15, 8, 19, 'Hi, there is a pharmacy about two miles away on Elm Street. I can give you directions if you need.', '2024-01-19 00:00:00'),
sql>-- (20, 18, 1, 20, 'Hi, thank you so much! I really appreciate your help. I will let you know if I need anything else.', '2024-01-20 00:00:00');
INSERT 0 20
sql>SELECT * FROM messages ;
message_id | guest_id | host_id | listing_id | message_text | timestamp
-----
1 | 2 | 1 | 1 | Hi, I just checked in. The place looks great! | 2024-01-01 00:00:00
2 | 6 | 1 | 2 | Hi, thanks for staying with me! I am glad you like it. Let me know if you need anything. | 2024-01-02 00:00:00
3 | 9 | 1 | 3 | Hi, I was wondering if you could tell me where the nearest grocery store is? | 2024-01-03 00:00:00
4 | 12 | 17 | 4 | Hi, sure thing! There is a grocery store about a mile away on Main Street. | 2024-01-04 00:00:00
5 | 15 | 13 | 5 | Hi, I also wanted to let you know that I will be checking out a little early tomorrow. | 2024-01-05 00:00:00
6 | 18 | 17 | 6 | Hi, no problem. Just let me know if you need any help with that. | 2024-01-06 00:00:00
7 | 20 | 1 | 7 | Hi, I was wondering if it is okay if I have a guest over tonight? | 2024-01-07 00:00:00
8 | 2 | 14 | 8 | Hi, sure, just let me know who it is and what time they will be arriving. | 2024-01-08 00:00:00
9 | 6 | 14 | 9 | Hi, I am also curious if there is a good place to get breakfast around here? | 2024-01-09 00:00:00
10 | 9 | 1 | 10 | Hi, yes, there is a great little cafe called The Daily Grind about a block away. | 2024-01-10 00:00:00
11 | 12 | 5 | 11 | Hi, I just wanted to let you know that I am checking out now. | 2024-01-11 00:00:00
12 | 15 | 5 | 12 | Hi, thanks for staying here! I hope you had a great stay. Let me know if you have any feedback. | 2024-01-12 00:00:00
13 | 18 | 8 | 13 | Hi, I will leave a review for you soon. I really enjoyed my stay! | 2024-01-13 00:00:00
14 | 20 | 13 | 14 | Hi, thank you! I am glad you had a good stay. I will look forward to your review. | 2024-01-14 00:00:00
15 | 2 | 17 | 15 | Hi, I just wanted to let you know that I had a great time and I will definitely be back! | 2024-01-15 00:00:00
16 | 6 | 14 | 16 | Hi, thank you! I am glad to hear that. I hope to see you again soon! | 2024-01-16 00:00:00
17 | 9 | 17 | 17 | Hi, I also wanted to let you know that I left a little something for you as a thank you. | 2024-01-17 00:00:00
18 | 12 | 5 | 18 | Hi, I was wondering if you could tell me where the nearest pharmacy is? I need to pick up some medication. | 2024-01-18 00:00:00
19 | 15 | 8 | 19 | Hi, there is a pharmacy about two miles away on Elm Street. I can give you directions if you need. | 2024-01-19 00:00:00
20 | 18 | 1 | 20 | Hi, thank you so much! I really appreciate your help. I will let you know if I need anything else. | 2024-01-20 00:00:00
(20 rows)
```

19 Creating table Payments and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Payments (payment_id, booking_id, amount, payment_date, payment_method) VALUES
sql_mart=# (1, 1, 100.00, '2024-01-01 00:00:00', 'Credit Card'),
sql_mart=# (2, 2, 200.00, '2024-01-02 00:00:00', 'PayPal'),
sql_mart=# (3, 3, 300.00, '2024-01-03 00:00:00', 'Credit Card'),
sql_mart=# (4, 4, 400.00, '2024-01-04 00:00:00', 'Bank Transfer'),
sql_mart=# (5, 5, 500.00, '2024-01-05 00:00:00', 'Credit Card'),
sql_mart=# (6, 6, 600.00, '2024-01-06 00:00:00', 'PayPal'),
sql_mart=# (7, 7, 700.00, '2024-01-07 00:00:00', 'Credit Card'),
sql_mart=# (8, 8, 800.00, '2024-01-08 00:00:00', 'Bank Transfer'),
sql_mart=# (9, 9, 900.00, '2024-01-09 00:00:00', 'Credit Card'),
sql_mart=# (10, 10, 1000.00, '2024-01-10 00:00:00', 'PayPal'),
sql_mart=# (11, 11, 1100.00, '2024-01-11 00:00:00', 'Credit Card'),
sql_mart=# (12, 12, 1200.00, '2024-01-12 00:00:00', 'Bank Transfer'),
sql_mart=# (13, 13, 1300.00, '2024-01-13 00:00:00', 'Credit Card'),
sql_mart=# (14, 14, 1400.00, '2024-01-14 00:00:00', 'PayPal'),
sql_mart=# (15, 15, 1500.00, '2024-01-15 00:00:00', 'Credit Card'),
sql_mart=# (16, 16, 1600.00, '2024-01-16 00:00:00', 'Bank Transfer'),
sql_mart=# (17, 17, 1700.00, '2024-01-17 00:00:00', 'Credit Card'),
sql_mart=# (18, 18, 1800.00, '2024-01-18 00:00:00', 'PayPal'),
sql_mart=# (19, 19, 1900.00, '2024-01-19 00:00:00', 'Credit Card'),
sql_mart=# (20, 20, 2000.00, '2024-01-20 00:00:00', 'Bank Transfer');
INSERT 0 20
sql_mart=# SELECT * FROM payments ;
 payment_id | booking_id | amount | payment_date | payment_method
-----+-----+-----+-----+-----
      1 |         1 | 100.00 | 2024-01-01 00:00:00 | Credit Card
      2 |         2 | 200.00 | 2024-01-02 00:00:00 | PayPal
      3 |         3 | 300.00 | 2024-01-03 00:00:00 | Credit Card
      4 |         4 | 400.00 | 2024-01-04 00:00:00 | Bank Transfer
      5 |         5 | 500.00 | 2024-01-05 00:00:00 | Credit Card
      6 |         6 | 600.00 | 2024-01-06 00:00:00 | PayPal
      7 |         7 | 700.00 | 2024-01-07 00:00:00 | Credit Card
      8 |         8 | 800.00 | 2024-01-08 00:00:00 | Bank Transfer
      9 |         9 | 900.00 | 2024-01-09 00:00:00 | Credit Card
     10 |        10 | 1000.00 | 2024-01-10 00:00:00 | PayPal
     11 |        11 | 1100.00 | 2024-01-11 00:00:00 | Credit Card
     12 |        12 | 1200.00 | 2024-01-12 00:00:00 | Bank Transfer
     13 |        13 | 1300.00 | 2024-01-13 00:00:00 | Credit Card
     14 |        14 | 1400.00 | 2024-01-14 00:00:00 | PayPal
     15 |        15 | 1500.00 | 2024-01-15 00:00:00 | Credit Card
     16 |        16 | 1600.00 | 2024-01-16 00:00:00 | Bank Transfer
     17 |        17 | 1700.00 | 2024-01-17 00:00:00 | Credit Card
     18 |        18 | 1800.00 | 2024-01-18 00:00:00 | PayPal
     19 |        19 | 1900.00 | 2024-01-19 00:00:00 | Credit Card
     20 |        20 | 2000.00 | 2024-01-20 00:00:00 | Bank Transfer
(20 rows)
```


20 Creating table Complaints and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Complaints (complaint_id, user_id, booking_id, complaint_text, timestamp, status) VALUES
sql_mart=# (1, 1, 1, 'Complaint 1', '2024-01-01 00:00:00', 'Open'),
sql_mart=# (2, 1, 2, 'Complaint 2', '2024-01-02 00:00:00', 'Resolved'),
sql_mart=# (3, 1, 3, 'Complaint 3', '2024-01-03 00:00:00', 'Open'),
sql_mart=# (4, 4, 4, 'Complaint 4', '2024-01-04 00:00:00', 'Resolved'),
sql_mart=# (5, 5, 5, 'Complaint 5', '2024-01-05 00:00:00', 'Open'),
sql_mart=# (6, 6, 6, 'Complaint 6', '2024-01-06 00:00:00', 'Resolved'),
sql_mart=# (7, 7, 7, 'Complaint 7', '2024-01-07 00:00:00', 'Open'),
sql_mart=# (8, 8, 8, 'Complaint 8', '2024-01-08 00:00:00', 'Resolved'),
sql_mart=# (9, 9, 9, 'Complaint 9', '2024-01-09 00:00:00', 'Open'),
sql_mart=# (10, 10, 10, 'Complaint 10', '2024-01-10 00:00:00', 'Resolved'),
sql_mart=# (11, 11, 11, 'Complaint 11', '2024-01-11 00:00:00', 'Open'),
sql_mart=# (12, 12, 12, 'Complaint 12', '2024-01-12 00:00:00', 'Resolved'),
sql_mart=# (13, 13, 13, 'Complaint 13', '2024-01-13 00:00:00', 'Open'),
sql_mart=# (14, 14, 14, 'Complaint 14', '2024-01-14 00:00:00', 'Resolved'),
sql_mart=# (15, 15, 15, 'Complaint 15', '2024-01-15 00:00:00', 'Open'),
sql_mart=# (16, 16, 16, 'Complaint 16', '2024-01-16 00:00:00', 'Resolved'),
sql_mart=# (17, 17, 17, 'Complaint 17', '2024-01-17 00:00:00', 'Open'),
sql_mart=# (18, 18, 18, 'Complaint 18', '2024-01-18 00:00:00', 'Resolved'),
sql_mart=# (19, 19, 19, 'Complaint 19', '2024-01-19 00:00:00', 'Open'),
sql_mart=# (20, 20, 20, 'Complaint 20', '2024-01-20 00:00:00', 'Resolved');
INSERT 0 20
sql_mart=# SELECT * FROM complaints ;
```

complaint_id	user_id	booking_id	complaint_text	timestamp	status
1	1	1	Complaint 1	2024-01-01 00:00:00	Open
2	1	2	Complaint 2	2024-01-02 00:00:00	Resolved
3	1	3	Complaint 3	2024-01-03 00:00:00	Open
4	4	4	Complaint 4	2024-01-04 00:00:00	Resolved
5	5	5	Complaint 5	2024-01-05 00:00:00	Open
6	6	6	Complaint 6	2024-01-06 00:00:00	Resolved
7	7	7	Complaint 7	2024-01-07 00:00:00	Open
8	8	8	Complaint 8	2024-01-08 00:00:00	Resolved
9	9	9	Complaint 9	2024-01-09 00:00:00	Open
10	10	10	Complaint 10	2024-01-10 00:00:00	Resolved
11	11	11	Complaint 11	2024-01-11 00:00:00	Open
12	12	12	Complaint 12	2024-01-12 00:00:00	Resolved
13	13	13	Complaint 13	2024-01-13 00:00:00	Open
14	14	14	Complaint 14	2024-01-14 00:00:00	Resolved
15	15	15	Complaint 15	2024-01-15 00:00:00	Open
16	16	16	Complaint 16	2024-01-16 00:00:00	Resolved
17	17	17	Complaint 17	2024-01-17 00:00:00	Open
18	18	18	Complaint 18	2024-01-18 00:00:00	Resolved
19	19	19	Complaint 19	2024-01-19 00:00:00	Open
20	20	20	Complaint 20	2024-01-20 00:00:00	Resolved

(20 rows)

21 Creating table Cleaning_Schedule and fill it with 20 entries

```
CREATE TABLE
sql_mart=#
sql_mart=# INSERT INTO Cleaning_Schedule (schedule_id, listing_id, cleaning_date, status) VALUES
sql_mart=# (1, 1, '2024-01-01 00:00:00', 'Scheduled'),
sql_mart=# (2, 2, '2024-01-02 00:00:00', 'Completed'),
sql_mart=# (3, 3, '2024-01-03 00:00:00', 'Scheduled'),
sql_mart=# (4, 4, '2024-01-04 00:00:00', 'Completed'),
sql_mart=# (5, 5, '2024-01-05 00:00:00', 'Scheduled'),
sql_mart=# (6, 6, '2024-01-06 00:00:00', 'Completed'),
sql_mart=# (7, 7, '2024-01-07 00:00:00', 'Scheduled'),
sql_mart=# (8, 8, '2024-01-08 00:00:00', 'Completed'),
sql_mart=# (9, 9, '2024-01-09 00:00:00', 'Scheduled'),
sql_mart=# (10, 10, '2024-01-10 00:00:00', 'Completed'),
sql_mart=# (11, 11, '2024-01-11 00:00:00', 'Scheduled'),
sql_mart=# (12, 12, '2024-01-12 00:00:00', 'Completed'),
sql_mart=# (13, 13, '2024-01-13 00:00:00', 'Scheduled'),
sql_mart=# (14, 14, '2024-01-14 00:00:00', 'Completed'),
sql_mart=# (15, 15, '2024-01-15 00:00:00', 'Scheduled'),
sql_mart=# (16, 16, '2024-01-16 00:00:00', 'Completed'),
sql_mart=# (17, 17, '2024-01-17 00:00:00', 'Scheduled'),
sql_mart=# (18, 18, '2024-01-18 00:00:00', 'Completed'),
sql_mart=# (19, 19, '2024-01-19 00:00:00', 'Scheduled'),
sql_mart=# (20, 20, '2024-01-20 00:00:00', 'Completed');
INSERT 0 20
sql_mart=# SELECT * FROM cleaning_schedule ;
 schedule_id | listing_id |   cleaning_date   | status
-----+-----+-----+-----
      1 |      1 | 2024-01-01 00:00:00 | Scheduled
      2 |      2 | 2024-01-02 00:00:00 | Completed
      3 |      3 | 2024-01-03 00:00:00 | Scheduled
      4 |      4 | 2024-01-04 00:00:00 | Completed
      5 |      5 | 2024-01-05 00:00:00 | Scheduled
      6 |      6 | 2024-01-06 00:00:00 | Completed
      7 |      7 | 2024-01-07 00:00:00 | Scheduled
      8 |      8 | 2024-01-08 00:00:00 | Completed
      9 |      9 | 2024-01-09 00:00:00 | Scheduled
     10 |     10 | 2024-01-10 00:00:00 | Completed
     11 |     11 | 2024-01-11 00:00:00 | Scheduled
     12 |     12 | 2024-01-12 00:00:00 | Completed
     13 |     13 | 2024-01-13 00:00:00 | Scheduled
     14 |     14 | 2024-01-14 00:00:00 | Completed
     15 |     15 | 2024-01-15 00:00:00 | Scheduled
     16 |     16 | 2024-01-16 00:00:00 | Completed
     17 |     17 | 2024-01-17 00:00:00 | Scheduled
     18 |     18 | 2024-01-18 00:00:00 | Completed
     19 |     19 | 2024-01-19 00:00:00 | Scheduled
     20 |     20 | 2024-01-20 00:00:00 | Completed
(20 rows)
```

22 Selecting employee_id, employee_name, manager_id and manager_name from Table Employees to verify recursive relationship

```
sql_mart=# SELECT
sql_mart=#     e1.employee_id AS Employee_ID,
sql_mart=#     e1.name AS Employee_Name,
sql_mart=#     e2.employee_id AS Manager_ID,
sql_mart=#     e2.name AS Manager_Name
sql_mart=# FROM
sql_mart=#     Employee e1
sql_mart=# LEFT JOIN
sql_mart=#     Employee e2 ON e1.manager_id = e2.employee_id;
```

employee_id	employee_name	manager_id	manager_name
1	Jessica		
2	Karen	1	Jessica
3	Tim	1	Jessica
4	Lucas	1	Jessica
5	Thomas	1	Jessica
6	Darl	1	Jessica
7	Sam	1	Jessica
8	Fritz	1	Jessica
9	Howard	1	Jessica
10	Sarah	1	Jessica
11	Aiko	1	Jessica
12	Huesna	1	Jessica
13	Barbara	1	Jessica
14	Kevin	1	Jessica
15	Robert	1	Jessica
16	Stefan	1	Jessica
17	Ramona	1	Jessica
18	Anna	1	Jessica
19	Jeff	1	Jessica
20	Leo	1	Jessica

(20 rows)

23 Selecting guest_id, host_name, listing_id, listing_location, host_id and host_name from Tables Bookings, Users and Listings to verify triple relationship

```

sql_mart=# SELECT
sql_mart=#     g.user_id AS Guest_ID,
sql_mart=#     g.username AS Guest_Name,
sql_mart=#     l.listing_id AS Listing_ID,
sql_mart=#     l.location AS Listing_Location,
sql_mart=#     h.user_id AS Host_ID,
sql_mart=#     h.username AS Host_Name
sql_mart=# FROM
sql_mart=#     Bookings b
sql_mart=# JOIN
sql_mart=#     Users g ON b.guest_id = g.user_id
sql_mart=# JOIN
sql_mart=#     Listings l ON b.listing_id = l.listing_id
sql_mart=# JOIN
sql_mart=#     Users h ON l.host_id = h.user_id;
 guest_id | guest_name | listing_id | listing_location | host_id | host_name
-----+-----+-----+-----+-----+-----
      2 | user2      |          1 | Paris, France    |        1 | user1
      6 | user6      |          1 | Paris, France    |        1 | user1
      9 | user9      |          1 | Paris, France    |        1 | user1
     12 | user12     |          4 | Amsterdam, Netherlands |      17 | user17
     15 | user15     |          5 | Prague, Czech Republic |      13 | user13
     18 | user18     |          6 | Dublin, Ireland  |      17 | user17
     20 | user20     |          7 | Berlin, Germany  |         1 | user1
      2 | user2      |          8 | Athens, Greece   |      14 | user14
      6 | user6      |          9 | Vienna, Austria  |      14 | user14
      9 | user9      |         10 | Edinburgh, Scotland |         1 | user1
     12 | user12     |         11 | Budapest, Hungary |         5 | user5
     15 | user15     |         12 | Copenhagen, Denmark |         5 | user5
     18 | user18     |         13 | Florence, Italy   |         8 | user8
     20 | user20     |         14 | Krakow, Poland    |        13 | user13
      2 | user2      |         15 | Lisbon, Portugal  |        17 | user17
      6 | user6      |         16 | Stockholm, Sweden |        14 | user14
      9 | user9      |         17 | Dubrovnik, Croatia |        17 | user17
     12 | user12     |         18 | Brussels, Belgium |         5 | user5
     15 | user15     |         19 | Reykjavik, Iceland |         8 | user8
     18 | user18     |         20 | Zurich, Switzerland |         1 | user1
(20 rows)

```


24 Selecting listing_id, location, price, username, date and availability_status from Tables Users, Listings and Availability_Calendar to verify triple relationship

```
sql_mart=# SELECT l.listing_id, l.location, l.price, u.username, ac.date, ac.availability_status
sql_mart=# FROM Listings l
sql_mart=# JOIN Users u ON l.host_id = u.user_id
sql_mart=# JOIN Availability_Calendar ac ON l.listing_id = ac.listing_id
sql_mart=# ORDER BY l.listing_id, ac.date;
```

listing_id	location	price	username	date	availability_status
1	Paris, France	150.00	user1	2024-10-15 08:30:00	Available
1	Paris, France	150.00	user1	2024-10-16 10:45:00	Booked
1	Paris, France	150.00	user1	2024-10-17 12:15:00	Available
4	Amsterdam, Netherlands	300.00	user17	2024-10-18 14:30:00	Available
5	Prague, Czech Republic	120.00	user13	2024-10-19 16:45:00	Booked
6	Dublin, Ireland	400.00	user17	2024-10-20 09:00:00	Available
7	Berlin, Germany	130.00	user1	2024-10-21 11:30:00	Available
8	Athens, Greece	280.00	user14	2024-10-22 13:45:00	Booked
9	Vienna, Austria	190.00	user14	2024-10-23 15:00:00	Available
10	Edinburgh, Scotland	110.00	user1	2024-10-24 17:15:00	Available
11	Budapest, Hungary	170.00	user5	2024-10-25 19:30:00	Booked
12	Copenhagen, Denmark	220.00	user5	2024-10-26 08:45:00	Available
13	Florence, Italy	350.00	user8	2024-10-27 10:00:00	Available
14	Krakow, Poland	160.00	user13	2024-10-28 12:30:00	Booked
15	Lisbon, Portugal	500.00	user17	2024-10-29 14:45:00	Available
16	Stockholm, Sweden	270.00	user14	2024-10-30 17:00:00	Available
17	Dubrovnik, Croatia	140.00	user17	2024-10-31 19:15:00	Booked
18	Brussels, Belgium	320.00	user5	2024-11-01 08:30:00	Available
19	Reykjavik, Iceland	200.00	user8	2024-11-02 10:45:00	Available
20	Zurich, Switzerland	180.00	user1	2024-11-03 12:00:00	Booked

(20 rows)

25 Selecting booking_id, payment_status, review_text, rating and username from Tables Booking, Reviews and Users to verify triple relationship

```
sql_mart=# SELECT b.booking_id, b.payment_status, r.review_text, r.rating, u.username
sql_mart=# FROM Bookings b
sql_mart=# JOIN Users u ON b.guest_id = u.user_id
sql_mart=# JOIN Reviews r ON b.booking_id = r.booking_id
sql_mart=# ORDER BY b.booking_id;
```

booking_id	payment_status	review_text	rating	username
1	Paid	Absolutely loved the place, felt like a hidden gem!	5	user2
2	Pending	Decent spot, but could use a bit more character.	3	user6
3	Paid	Incredible stay, exceeded all expectations!	5	user9
4	Paid	Breathtaking views, a truly serene experience.	4	user12
5	Pending	Average stay, nothing too remarkable.	3	user15
6	Paid	Outstanding accommodation, felt like a dream!	5	user18
7	Paid	Cozy and inviting, perfect for a relaxing break.	4	user20
8	Pending	Bang for the buck, definitely worth it.	4	user2
9	Paid	Charming place, every detail was delightful.	5	user6
10	Paid	Meh experience, didn't quite hit the mark.	3	user9
11	Pending	Quirky and delightful, a stay to remember!	5	user12
12	Paid	Spotless property, cleanliness was top-notch.	4	user15
13	Paid	Exceptional service, made me feel right at home.	5	user18
14	Pending	Fair stay, had its ups and downs.	3	user20
15	Paid	Luxurious vibes, felt like royalty!	5	user2
16	Paid	Tranquil retreat, perfect for unwinding.	4	user6
17	Pending	Good location, but amenities could be better.	3	user9
18	Paid	Top-notch stay, highly recommend to all!	5	user12
19	Paid	Quaint and charming, a true hidden treasure.	4	user15
20	Pending	Comfortable stay, but lacked that special touch.	3	user18

(20 rows)

Abstract

This document provides a detailed overview of the database management functionality for an Airbnb-like platform. The platform facilitates the rental of apartments and bedrooms, connecting hosts, guests and admins through an SQL database management system. The database is designed using a state-of-the-art PostgreSQL 15-based management system, ensuring efficient storage, retrieval, and management of data. Furthermore, this document highlights the key aspects of the database schema, and metadata stored within the system. The database is normalized to optimize performance and maintain data integrity.

Database Management Functionality

The database management system supports the following functionalities:

User Management

- Registration and Authentication: Users are registered by providing necessary details like names and e-mail address.
- Platform policies: Admins can provide detailed platform policies that can be viewed by users and hosts.

Listing Management

- Creating Listings: Hosts can create new property listings with detailed descriptions, amenities and photos.
- Updating Listings: Hosts can update or delete their existing listings.
- Search and filter: Guests can search for listings based on various criteria like location, neighborhood, price, and ratings.
- Booking availability: Hosts can manage accommodation availability through a cleaning schedule and availability calendar.
- Wishlist: Guests can save accommodations via wishlists and process their bookings at different time points.
- House rules: Hosts can provide detailed house rules for their accommodations that can be viewed by guests.
- Cancellation policies: Hosts can provide cancellation policies that can be viewed by guests.

Booking and Payment Management

- Booking: Guests can book available listings.

- **Payment Processing:** The system processes payments through integrated payment status and further payment details.

Review and Rating System

- **Review Submission:** Guests and hosts can submit reviews after a booking is completed.
- **Messages:** Hosts and guests can send messages related to their bookings and experiences.
- **Complaints:** Guests and hosts can submit complaints about accommodations, services or their stay.

Metadata

The metadata stored in the system includes:

- 20 tables with a total of 151 columns, 414 entries and a database size about 8606511 bytes.
- Users table with 4 columns, 1 primary key and 34 entries.
- Employee table with 4 columns, 1 primary key, 2 foreign keys and 20 entries.
- Listings table with 6 columns, 1 primary key, 1 foreign key and 20 entries.
- Bookings table with 5 columns, 1 primary key, 2 foreign keys and 20 entries.
- Reviews table with 6 columns, 1 primary key, 3 foreign keys and 20 entries.
- Availability_Calendar with 3 columns, 1 primary key, 1 foreign key and 20 entries.
- Platform_Policies table with 2 columns, 1 primary key and 20 entries.
- Super_Host_Status table with 2 columns, 1 primary key, 1 foreign key and 20 entries.
- User_Reviews table with 2 columns, 2 foreign keys and 20 entries.
- User_Policies table with 2 columns, 1 primary key, 1 foreign key and 20 entries.
- Cancellation_Policy table with 3 columns, 1 primary key, 1 foreign key and 20 entries.
- Wish_List table with 3 columns, 1 primary key, 2 foreign keys and 20 entries.
- Amenities table with 3 columns, 1 primary key, 1 foreign key and 20 entries.
- House_Rules table with 3 columns, 1 primary key, 1 foreign key and 20 entries.
- Neighborhood table with 3 columns, 1 primary key, 1 foreign key and 20 entries.
- Location table with 7 columns, 1 primary key, 1 foreign key and 20 entries.
- Messages table with 6 columns, 1 primary key, 3 foreign keys and 20 entries.
- Payments table with 5 columns, 1 primary key, 1 foreign key and 20 entries.
- Complaints table with 6 columns, 1 primary key, 1 foreign key and 20 entries.
- Cleaning_Schedule table with 4 columns, 1 primary key, 1 foreign key and 20 entries.